```
depot> sqlite3 -line db/development.sqlite3 "select * from accounts"
     id = 1
 number = 12345
balance = 100

     id = 2
 number = 54321
balance = 200
```

However, there's a trap waiting for you here. The transaction protected the database from becoming inconsistent, but what about our model objects? To see what happened to them, we have to arrange to intercept the exception to allow the program to continue running:

rails7/e1/ar/transactions.rb

```
peter = Account.create(balance: 100, number: "12345")
paul  = Account.create(balance: 200, number: "54321")
```

rails7/e1/ar/transactions.rb

```
begin
  Account.transaction do
    paul.deposit(350)
    peter.withdraw(350)
  end
rescue
  puts "Transfer aborted"
end

puts "Paul has #{paul.balance}"
puts "Peter has #{peter.balance}"
```

What we see is a little surprising:

```
Transfer aborted
Paul has 550.0
Peter has -250.0
```

Although the database was left unscathed, our model objects were updated anyway. This is because Active Record wasn't keeping track of the before and after states of the various objects—in fact, it couldn't, because it had no easy way of knowing just which models were involved in the transactions.